

RESEARCH ARTICLE

Uncertainty-Aware Reinforcement Learning for Portfolio Optimization

BAYARAA ENKHSAIKHAN¹, (Student Member, IEEE), AND **OHYUN JO²**, (Member, IEEE)¹Department of Computer Science, Chungbuk National University, Cheongju 28644, South Korea

Corresponding author: Ohyun Jo (ohyunjo@chungbuk.ac.kr)

This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) funded by the Korean Government [Ministry of Science and ICT (MSIT)] through the Development of 5G+ Intelligent Base-Station Software Modem under Grant 2021-0-00165, and in part by the National Research Foundation of Korea (NRF) funded by the Korean Government (MSIT) under Grant 2021R1A2C2095289.

ABSTRACT We explored the use of Reinforcement Learning (RL) combined with risk assessment for optimizing investment portfolios. The dynamic nature of trading, compounded by market frictions, the responses of other market participants, and uncertainties, poses challenges to portfolio optimization. The financial market's intricacies make it difficult to model accurately, compounded by regulatory requirements and internal risk policies mandating risk-averse decisions to avoid catastrophic outcomes. To address this, we proposed risk estimation for investor's risk tolerance threshold. Moreover, modern Deep Learning models are adept at approximating complex relationship between abundant data, however, the main drawback we face now a day is generalization of the relationship to the unseen data. Therefore, the epistemic uncertainty can pose risk to the decision making system. This uncertainty is further addressed using a Variational Autoencoder (VAE) to estimate, and Cost Network to backpropagate riskiness through the model to learn actions with safe results. The actions with stable result or lower reward will be avoided due to reward optimization of RL. Consequently, we successfully managed to reduce the risk and uncertainties in the agent testing process. Our risk-constrained RL algorithm demonstrated zero violation of the constraint in the testing phase. This suggests that adopting a risk-averse RL approach could be beneficial for portfolio optimization, particularly for risk-averse investors.

INDEX TERMS Risk-averse reinforcement learning, portfolio optimization, variational auto encoder.

I. INTRODUCTION

Financial portfolio optimization is a strategy used by investors to construct an investment portfolio that maximizes expected returns while minimizing risk. The core idea is to allocate investments across various assets in a way that balances risk and return according to the investor's preferences and objectives. However, the selection of the optimal portfolio for investment should be bounded by the investor's risk tolerance and capacity. Risk appetite is typically predefined for sophisticated investors, while regulatory bodies may also impose risk constraints on investors.

The associate editor coordinating the review of this manuscript and approving it for publication was Jiachen Yang³.

The anticipated risk and return of portfolios remain fluid, subject to change over time. Consequently, investors must actively engage in market trading and adapt their portfolios accordingly. However, ongoing trading entails fees and associated costs, which may surpass the returns generated from portfolio trading. Conventional portfolio optimization techniques rely on financial models, which offer detailed explanations and rationale. However, these heuristic models typically assume that factors beyond those explicitly inputted into the models remain fixed. To address this limitation, deep learning methods offer enhanced flexibility. With a sufficiently large dataset, neural networks can approximate various types of functional relationships. Particularly noteworthy is the Deep Reinforcement Learning (DRL) algorithm, which is capable of tackling continuous

decision-making challenges and its unsupervised nature due to absence of target output data, making it suitable for dynamic portfolio optimization tasks. Nonetheless, it's essential to exercise caution when employing deep learning models due to the risk of overfitting [1]. This can be tackled by implementing a model uncertainty solution.

We propose the implementation of the Risk-averse Reinforcement Learning model. Our assessment of risk revolves around the concept of uncertainty, which introduces potential risks into the decision-making process. These uncertainties stem from both the inherent nature of the data and the predictive capabilities of the model, categorized as aleatoric uncertainties (attributed to data stochasticity) and epistemic uncertainties (stemming from model uncertainty) within the realm of deep learning. With such improvements, a model can learn actions with more certain results. Actions with relatively low rewards but certain results are avoided by the RL algorithm. In financial markets, significant fluctuations can arise due to market expectations, representing aleatoric uncertainty where precise outcomes become increasingly difficult to predict. Furthermore, limitations in knowledge and data availability contribute to a decrease in model prediction accuracy.

To address the both types of uncertainty, we employed variance prediction along with the expected output, which accommodates aleatoric uncertainty. Moreover, we utilized VAE to characterize model uncertainties for out-of-distribution data. Our contribution is that we introduced a cost network to quantify aleatoric uncertainty and a VAE to quantify epistemic uncertainty, complementing the Actor-Critic model [2]. However, avoiding both types of risk/uncertainty entirely can be inefficient, as it limits the model's exploration capacity. Thus, the cost network's gradient back-propagation influenced the actor network to learn safe actions when risk breached certain constraints, along with the critic network's gradient to maximize expected rewards. Moreover, the epistemic uncertainty estimation provided confidence for the cost network; as epistemic uncertainty decreased, predictions became more accurate to improve model performance.

II. LITERATURE REVIEW

Our RL algorithm in this paper is uncertainty aware, constrained and for portfolio optimization. Hence, we conducted research in the following related fields.

A. UNCERTAINTY-AWARE RL METHOD

Numerous papers have been written on uncertainty awareness. Reference [3] has surveyed on the uncertainty aware deep neural networks, and categorized the methods as follows: i) Single deterministic methods; ii) Bayesian methods; iii) Ensemble methods; and iv) Test-time augmentation methods. Single deterministic methods cannot capture the both types of uncertainty because they are typically trained to learn aleatoric uncertainty in regression tasks. Some works [4], [5] focus on detecting epistemic uncertainty in out-of-distribution (OOD) data and quantifying it by measuring

the distance between OOD data and in-distribution data. The Bayesian methods involve Bayesian Neural Network and measure the both types of uncertainty. However, Bayesian methods have a much more complicated training process, and the outcome highly depends on the prior belief, which cannot be estimated or trained. Moreover, Ensemble methods also measures the both uncertainties but require training multiple models with different parameter initialization. Reference [6] discovered that uncertainty can be approximated using Monte-Carlo dropout technique in deep learning. The test-time augmentation approach involves generating numerous variations of each test sample through data augmentation techniques, then evaluating all these samples to calculate a predictive distribution for assessing uncertainty.

However, the above methods, except Single deterministic method, require sampling to quantify uncertainty, making it impossible to backpropagate the uncertainty through the model even though they are effective at detecting both types of uncertainty. Hence, we use Single deterministic method to quantify and backpropagate the aleatoric uncertainty and VAE for epistemic uncertainty.

B. RISK-AVERSE/SAFE RL METHOD

Risk-averse RL algorithms are implemented with hard, soft, or probabilistic safety criteria [7]. Many works have focused on risk-averse RL tasks. Some studies, such as [8] and [9], use worst-case risk measure as objective function, therefore, they directly reduce risk. However, as noted by [10], this approach prevents the agent from ever purchasing risky assets. Other works, such as [11], introduce penalty to reward function for uncertainty, penalizing the agent for constraint violations. This makes the agent risk-averse in policy optimization, and since the reward is modified, this approach is referred to as utility optimization in some studies. However, it does not guarantee the agent will consistently make risk-constrained actions. Additionally, it has the drawback of not ensuring that the training process remains stationary [10].

Reference [12] used Lagrangian relaxation to transform a constrained optimization to an unconstrained one. In this hard-constrained case, the trading agent's actions are risk-constrained, hence the agent does not take any action exceeding the risk acceptance level. This type of Constrained Markov Decision Process (CMDP) is typically solved with Lagrangian multiplier to enforce the constrain on the objective function, ensuring adherence to the specified constraint function.

C. PORTFOLIO OPTIMIZATION

The portfolio optimization methods leveraging deep learning models offer the advantage of modeling complex behaviors and making decisions even in environments with high-dimensional states and actions [11]. Supervised modeling is a common application of deep learning in portfolio optimization. These models forecast asset prices/returns,

risk/variance, or both, which inform the portfolio's construction and re-balancing strategies. Reference [13] explores the use of deep learning for asset price prediction. The study demonstrates that deep neural networks (DNNs) can capture nonlinear relationships in financial data, leading to more accurate forecasts compared to traditional models. The improved predictive accuracy aids in constructing more efficient portfolios. Reference [14] focuses on sequence-to-sequence models for stock price prediction. The authors employ long short-term memory (LSTM) networks to capture temporal dependencies in financial time series data, resulting in superior predictive performance. This enhanced forecasting capability supports better portfolio optimization decisions.

Deep learning methods have significantly advanced portfolio optimization by enhancing predictive accuracy. However, these works using supervised learning techniques require unsupervised optimization due to lack of perfectly constructed portfolio as target to learn. Hence, reinforcement learning contributes to more effective portfolio management strategies. Continued research aims to refine these models for better adaptability and performance in dynamic financial markets. Recent studies, such as [15] and [16], have utilized RL to maximize returns or the risk-return ratio (Sharpe ratio), following the objective function suggested [17]. However, portfolio optimization is essentially a complex problem of utility maximization under constraints [18]. Consequently, these algorithms have proven to be less practical due to the varying risk appetites and capacities of different investors.

III. THEORETICAL BACKGROUND

A. PORTFOLIO OPTIMIZATION

Portfolio optimization is the process of selecting the best combination of financial assets (such as stocks, bonds, and other investments) to achieve specific investment goals, typically maximizing returns for a given level of risk or minimizing risk for a given level of return. This process involves balancing the trade-off between risk and return, considering the investor's risk tolerance, investment horizon, and other constraints.

Modern Portfolio Theory (MPT), developed by Markowitz [19], revolutionizes investment by optimizing portfolios to balance risk and return. Mean-Variance Optimization (MVO), also known as Markowitz portfolio optimization technique, was introduced by [19] and it assumes that the portfolio's expected return and risk can be represented by the historical mean and standard deviation. This method calculates the efficient frontier, representing portfolios that provide the best possible expected return for a given level of risk. However, this technique does not take the investor's risk tolerance or limit into consideration. In fact, investor's utility function, which balances expected return and risk to reflect their satisfaction, is an important factor. For institutional investors, risk tolerance is predefined by the internal risk policy or legislation. We expect that the

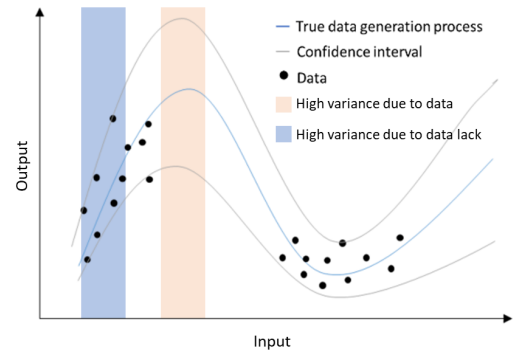


FIGURE 1. Model uncertainty on explored and unexplored areas.

investors are extremely risk-averse beyond the predefined risk constraint. Therefore, our problem is defined as follows:

$$\max_a E\left[\sum_t r_t a_t\right], \text{ subject to } VaR(s_t, a_t) < c \text{ for } \forall i \in [1, t] \quad (1)$$

In Eq. 1, the action and state observations are denoted by a and s . VaR is risk measurement, c is pre-defined constraint term, and t is episode represents time steps. The conventional models use historical mean returns and variance to identify optimal portfolios. This assumption is extended to any investors when optimizing portfolios at an aggregate level, ensuring that risk management aligns with broader financial strategies. MPT provides a robust foundation for constructing efficient portfolios, balancing risk and return to meet investor preferences. It remains a cornerstone of investment strategy, guiding both individual and institutional investors in achieving their financial goals.

B. UNCERTAINTY/RISK ESTIMATION

Uncertainty and risk estimation in deep learning are crucial for improving model reliability, particularly in high-stakes applications such as healthcare, autonomous driving, and finance. These techniques help quantifying the confidence of a model's predictions, thereby guiding better decision-making.

1) ALEATORIC UNCERTAINTY

This type of uncertainty arises from inherent noise in the data. It's irreducible and can be modeled using techniques like Bayesian neural networks or ensemble methods to estimate prediction variance.

2) EPISTEMIC UNCERTAINTY

This uncertainty is due to a lack of knowledge about the optimal model parameters. It can be reduced with more data and can be captured using Bayesian inference or Monte Carlo dropout, where multiple forward passes with dropout provide a distribution of outputs.

Uncertainty and risk estimation in deep learning enhance model interpretability and trustworthiness, enabling more

robust and safe deployment in various critical applications. By leveraging techniques like Bayesian approaches, Monte Carlo dropout, and ensemble methods, deep learning models can provide not just predictions, but also a measure of their confidence, leading to better and more reliable outcomes.

The uncertainty is due to inherent noise in the data (aleatoric) or incomplete knowledge about the environment (epistemic) [20]. Epistemic uncertainty represents the lack of exploration; hence, it can be used for efficient exploration:

$$\underbrace{V(y|s, a)}_{\text{Total}} = \underbrace{E[V(y - \Phi(s, a))]}_{\text{Epistemic}} + \underbrace{E[V(\Phi(s, a))]}_{\text{Aleatoric}} \quad (2)$$

V is the variance, E is the expectation, Φ is the prediction function, and y is the ground truth. In (2), the epistemic uncertainty is what the model cannot predict. However, given enough data is collected, the epistemic uncertainty tends to be lower, and the variance left in the ground truth must be due to the aleatoric uncertainty. Hence, the epistemic uncertainty is correlated with the exploration of the data [20]. Out-of-distribution (OOD) data, which is less explored data, tend to have greater epistemic uncertainty as shown in Fig. 1. We used a VAE for OOD detection as the epistemic uncertainty, then, a deterministic network to predict heteroskedastic variance as aleatoric uncertainty. This technique is widely used in the related studies, such as [21], [22], and [20]. It is more efficient to handle aleatoric and epistemic uncertainties differently only during the training phase [23], a process known as Bayesian optimization.

C. VARIATIONAL AUTOENCODER

VAE [24] is a type of generative model that learns to represent complex data distributions in a lower-dimensional latent space while allowing for stochastic generation of new data points. It consists of two primary components: the encoder and the decoder, along with a latent space that captures the learned features. The encoder takes the input data and maps it into a distributional latent space by outputting mean and variance of the latent variables. The output of the encoder represents the mean and variance of this distribution, allowing for uncertainty in the encoding process. The latent space is a lower-dimensional space where the encoder maps the input data. Each data point is represented as a random variable, sampled from the distribution defined by the mean and variance parameters learned by the encoder. This sampling is done using the reparameterization trick, which ensures that backpropagation is possible.

The decoder takes samples from the latent space and reconstructs the original data or generates new data. It tries to map these latent variables back to the original input space, producing an output as close as possible to the real data. The reconstruction is trained to minimize the difference between the input data and the reconstructed data using a loss function, such as the reconstruction loss. The total loss in a VAE is a combination of: i) Reconstruction Loss: Measures how well the decoder reconstructs the input data from the latent space

representation. ii) KL Divergence Loss: Encourages the latent space to be close to a standard normal distribution, ensuring smooth and meaningful latent space transitions for data generation. KL divergence measures the difference between the learned latent distribution and a prior distribution. The encoder learns a probabilistic latent space, and the decoder generates data from this space, allowing the VAE to learn representations that can capture uncertainty and generate new samples that resemble the original data.

D. EPISTEMIC RISK ASSESSMENT

We focus on inference of the states. Our critic network is trained with VAE, which uses variational distribution as inference. One of the most intriguing attribute of the VAE is that it can memorize frequently explored data instances and distinguish the OOD data [20]. Technically, the OOD is the reason behind epistemic uncertainty which is problematic to models due to lack of training data. Hence, the VAE predicts not only the distributional output, but also OOD score that can be used for aleatoric and epistemic uncertainty. OOD detection involves identifying data points that do not belong to the training data distribution. VAEs can be used for OOD detection based on the following concepts:

1) RECONSTRUCTION ERROR

The reconstruction error measures how well the VAE can reconstruct the input data from its latent representation. For in-distribution data, the VAE should be able to reconstruct the input data accurately, resulting in low reconstruction errors. For OOD data, the reconstruction error is typically higher because the VAE has not seen such data during training and struggles to reconstruct it accurately [25].

2) LATENT SPACE DISTRIBUTION

VAEs are trained to map in-distribution data to a latent space that follows a specific prior distribution (e.g., a standard normal distribution). By examining the distribution of latent space representations of new data points, one can determine whether they are in-distribution or OOD. If a data point's latent representation deviates significantly from the prior distribution, it is likely OOD [26].

3) COMBINING RECONSTRUCTION ERROR AND LATENT SPACE ANOMALIES

A more robust approach to OOD detection combines both reconstruction error and latent space anomalies. This involves checking both the reconstruction error and the likelihood of the latent representation under the prior distribution. Data points with high reconstruction errors and low likelihood in the latent space are flagged as OOD [21].

E. DEEP DETERMINISTIC POLICY GRADIENT

Deep Deterministic Policy Gradient (DDPG) [27] is a model-free, off-policy algorithm used in reinforcement learning for continuous action spaces. The DDPG algorithm leverages

deep neural networks to approximate both the policy and the value function.

1) POLICY NETWORK

The actor represents the policy $\pi(s)$, which maps states s to actions a . It is a deterministic function that outputs a single continuous action for a given state. The objective of the actor is to maximize the expected return by selecting actions that maximize the value of the current state. The actor is updated using the gradient of the Q-value with respect to the selected actions, guiding it to choose actions that maximize expected rewards.

2) VALUE NETWORK

The critic represents the Q-value $Q(s, a)$, which evaluates how good a given action a is in a particular state s . The critic learns to predict the time-discounted expectation on cumulative reward starting from state s , taking action a , and following the actor's policy thereafter. The critic is updated by minimizing the Temporal Difference (TD) error, which can be measured by the Bellman equation.

3) TARGET NETWORKS

DDPG uses target networks for both the actor and the critic. These are essentially copies of the actor and critic networks that are updated more slowly to stabilize training. The target actor and critic networks are used to calculate the target Q-value, reducing the likelihood of overestimating Q-values. These networks are updated by soft updates, where the parameters of the target networks are updated gradually.

4) EXPLORATION

DDPG requires an external source of exploration since it is an off-policy algorithm. A common technique is to add noise to the actions output by the actor during training. This noise is often generated using an Ornstein-Uhlenbeck process, which produces temporally correlated noise suitable for exploration in physical environments.

We delve deeper into each of the networks mentioned here with unique proposals from our side in the following sections.

F. ALEATORIC RISK ASSESSMENT

The risk is measured with uncertainty what we cannot predict precisely. The input to the variation prediction network is the dataset, and the output is heteroskedastic variance of the target, similar to [28]. Then, we calculate Value at Risk (VaR), which is widely adopted metric for assessing the risk of the portfolio in finance [29]. Let u be a return (percentage change in value) of an asset. The VaR at $\alpha \in (0, 1)$ confidence level is defined as (F_u is cumulative distribution function):

$$VaR_\alpha(u) = -\inf \{u \in R : F_u(u) > \alpha\} \quad (3)$$

According to a parametric method, known as variance-covariance method, VaR should be equal to the following

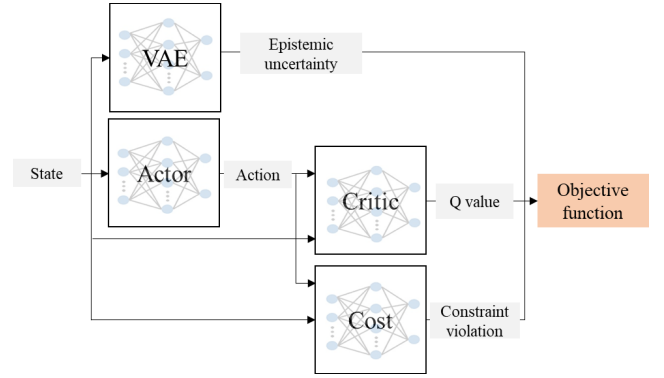


FIGURE 2. The feedforward process of the proposed model is illustrated. The grey boxes represent inputs and outputs, the white boxes represents deep neural networks, and orange box represents objective function to optimize the model.

given that the output is normally distributed:

$$VaR_\alpha(u) = z_\alpha \sigma \quad (4)$$

where z is z-score corresponding to the standard normal distribution at given confidence α , and σ is standard deviation. We can optimize variance prediction network using (4) where σ is the output. The portfolio VaR is (a is weight of assets in portfolio):

$$VaR_\alpha(u, a) = \sum_{i,j} a_i \cdot VaR_\alpha(u_i)^{0.5} \cdot a_j \cdot VaR_\alpha(u_j)^{0.5} \cdot cor(u_i, u_j) \quad (5)$$

IV. UNCERTAINTY AWARE RL METHOD

Portfolio optimization requires a longer-time horizon analysis and greater consideration of risk compared to speculative trading. Hence, we utilize fixed-time intervals data (daily data) to comply with with a Markov Decision Process (MDP). Actions are taken after observing market data, price information, as well as rewards that are dependent on the current state and chosen action. Thus, this dynamic decision-making task can be set up as an MDP.

The RL model learns by observing the state (input data), taking actions from a set of possible actions, and optimizing the policy for actions to be taken in specific states. Typically, the RL algorithm interacts with the environment, learning through continuous trial and error. The policy is evaluated based on the rewards received. Consequently, the concepts of exploration and exploitation are crucial, especially in risk-averse RL algorithms [30]. Exploitation involves taking actions that yield the highest reward according to the current policy, while exploration involves investigating less familiar states to discover potentially better policies.

A. ACTION

Our action space is weights for each asset in our portfolio. The weight corresponds to i -th asset at time t is $a_{i,t}$. The weights are constrained in the interval $[0; 1]$. The total value of the

portfolio is v_t . The decision step for MDP is one day, and the episode is a sequence of steps during a complete MDP that spans 250 business days in a year.

B. REWARD

The reward is portfolio value change at each day including dividend. We harness self-financing trade to use; therefore, the discount factor can be 1, which means that the final portfolio value is our objective. Hence, the immediate reward for step t is:

$$r_t = v_t - v_{t-1} - fee_t \quad (6)$$

The trading fee is calculated using ϕ as percentage constant at 0.25 percent and p as price:

$$fee_t = \phi \sum_i \left(\frac{a_{i,t} v_t p_{i,t-1}}{p_{i,t}} - a_{i,t-1} v_{t-1} \right) \quad (7)$$

C. STATE OBSERVATIONS

We have selected current and historical return information, trend indicators and normalized trading volume over various time horizons. The factors are important suggested by [31] and [32]. Moreover, the latest market data such as current market index, economic statistics are also used as input data.

D. REINFORCEMENT LEARNING ALGORITHM

Actor-Critic network [2] is utilized as model settings. The actor network selects an action after observing the state as an input. The Dirichlet distribution is used for the output layer of the actor. The critic network evaluates that action in the respective state. The Q-network/value network evaluates expected reward [33], given the policy.

E. VALUE NETWORK

DDPG [27] is chosen as optimization algorithm due to its suitability for continuous action spaces. The time-discounted expectation on reward is measured using the value network $Q(a_t, s_t)$. Due to continuous action space, a deterministic optimal action is predicted with policy network: $\pi(s_t) = \arg \max_a Q(s_t, a_t)$. As per Bellman equation, optimal value network must meet following condition:

$$Q(s, a) = r + \gamma \max_a Q(s', a') \quad (8)$$

Here, s' is next state, a' is next action, γ is discount factor for time step and r is the reward defined by Eq. 6. Replay buffer is used for sample efficiency [34]. Hence, the loss for the value network is TD error as mention earlier:

$$L_q = \frac{1}{N} \sum_{i=1}^N [Q(s, a) - (r + \gamma(1 - d) \max_a Q'(s', a'))]^2 \quad (9)$$

In order to find the optimal parameters of the value network, we minimize (9) through gradient update with respect to its parameters. The binary value $d \in \{0; 1\}$ indicates whether the current state is terminal. Q' network is the target network. During the gradient descent update,

the target network's parameters, $w^{Q'}$, are replaced with the weighted average of the parameters from the current target network and the updated value network according to following formula ($0 < \tau < 1$):

$$w^{Q'} \leftarrow \tau w^Q + (1 - \tau) w^{Q'} \quad (10)$$

F. VARIATIONAL AUTO ENCODER

For epistemic uncertainty, the VAE is used as an out-of-distributional data detector. It is crucial to detect OOD due to epistemic uncertainty that the model faces for precise prediction.

$$Loss_{vae} = Dis(s, D_{vae}(E_{vae}(s) \times [1, x_{x \sim N(0,1)}]^T)) + B_{KL}(E_{vae}(s), N(0, 1)) \quad (11)$$

Here, s is the state observations, E_{vae} and D_{vae} are the encoder and decoder networks, B_{KL} is the Kullback-Leibler divergence, Dis is the Euclidean distance as reconstruction error, x is a sample from normal distribution, and $\times$ is element-wise multiplication.

Modern deep learning models are excellent at approximating any kind of relation between input and output, however, they are less effective at generalizing the relation into broader contexts, such as OOD. Hence, it is crucial to estimate the reliability of the model result, especially for risk-averse and high-stake tasks. We used combination of the reconstruction error and latent space anomalies as the score for epistemic uncertainty:

$$u_{ep} = \delta B_{KL}[E_{vae}(s); N(0, 1)] + \lambda Dis[D_{vae}(\mu_s + 1.96\sigma_s); s] \quad (12)$$

Here, $Dis(D_{vae}(\mu_s + 1.96\sigma_s), s)$ is the Euclidean distance (as reconstruction error) between a sample with 95 percent confidence in latent space, and the original input as suggested by [35]. The parameters λ , δ , and the weights of the Euclidean distance are trainable. We considered two options: i) Dis is weighted Euclidean distance, and the weights are trainable (using logistic function used to confine the weights to be between 0 and 1); and ii) Dis is equally weighted Euclidean distance. No significant performance difference is observed between the two options (see Section VI for more). We chose equally weighted option to avoid overfitting.

Mean Absolute Error (MAE) is selected as the target value to train (12). As the OOD score indicated by (12) increases, the epistemic uncertainty should decrease. The frequently visited data have a lower OOD score because the VAE is better calibrated for such data, and the model tends to make fewer errors on data that it has seen more often. The value network should be $y_i \leftarrow r_i + \gamma(1 - d)Q'(s_{i+1}, \pi'(s_{i+1}))$. Hence, the loss to train the parameters of (12) is:

$$L_{ep} = \frac{1}{N} \sum_{i=1}^N (|y_i - Q(s_i, a_i)| - u_{ep}(s_i, a_i))^2 \quad (13)$$

G. CONSTRAINT TERM

The cost network, $c(s, a)$, which predicts VaR (3) is utilized to predict only aleatoric uncertainty. We trained the actor to avoid only the aleatoric risk to maintain exploration. if the actor were trained to avoid epistemic uncertainty, the network would avoid exploration altogether, preventing the model from converging. The excessive aleatoric risk beyond a predefined limit must be minimized, and the gradient is backpropagated to the policy network. The cost network, $c(s, a)$, used to backpropagate the excess risk through minimization, is:

$$C(s_i, a_i) = \begin{cases} 0, & \text{if } c(s_i, a_i) < \zeta \\ \frac{1}{1 + u_{ep}(s, a)} c(s_i, a_i), & \text{else} \end{cases} \quad (14)$$

Here, ζ represents the investor's risk appetite constraint. The term $(1/1 + u_{ep}(s, a))$ serves as confidence parameter, which lies in range $[0, 1]$ since $u_{ep}(s, a) \geq 0$. As more exploration is conducted, this confidence should increase. Equation (14) shows that the constraint violation is back-propagated to the agent until the constraint is met, otherwise, zero gradient is back-propagated. This equation plays a vital role in assessing the loss from the excess risk and propagating the loss back to the actor network. If the risk is within accepted level, no gradient is backpropagated.

H. POLICY NETWORK

The actor network, $\pi(s)$, is a deterministic network that chooses the optimal action. The optimal action must have the highest Q-value; hence, the objective function to maximize for the constrained agent is:

$$J_\pi = \frac{1}{N} \sum_{i=1}^N [Q(s, \pi(s)) - C(s, \pi(s))] \quad (15)$$

To optimize parameters of the policy network, (15) is maximized via gradient update with respect to the policy network's parameters while keeping the value network and cost networks' parameters frozen. The gradient flows through the value network to the policy, guiding it toward higher rewards. If the constraint is violated as per (14), then the gradient flows through the cost network, guiding the policy network toward safer actions. Moreover, the confidence term $(1/1 + u_{ep}(s, a))$ in (14) is considered constant during the policy optimization process. To provide exploration, the following distribution is used instead the Ornstein-Uhlenbeck process in DDPG:

$$Dir(v) = \frac{1}{B(v)} \prod_{i=1}^K x_i^{v_i-1} \quad (16)$$

Here, v is a vector, $B(v)$ is multivariate beta function:

$$B(v) = \frac{\prod_{i=1}^K \Gamma(v_i)}{\Gamma(\sum_{i=1}^K v_i)} \quad (17)$$

The shape of the distribution is determined by the vector v (v_i is a i -th element of the vector). $\Gamma(v)$ is gamma function. This Dirichlet policy can address some issues of DDPG since our action is the asset weights and sum up to 1 [36]. The shape determining vector v is calculated by:

$$v = \kappa \cdot \pi(s) \quad (18)$$

Here, κ ($\kappa > 1$) is hyperparameter that controls exploration, $\pi(s)$ is the initial output of the actor network before the exploration, and the final action is sampled from (16). An insufficient κ can hinder the model's convergence. During the testing phase, we do not need to add this noise to the action. Therefore, the mode of the Dirichlet distribution is used instead of sampling during testing.

The target network's parameters are updated through weighting:

$$w^{\pi'} \leftarrow \tau w^{\pi} + (1 - \tau) w^{\pi'} \quad (19)$$

I. TOTAL UNCERTAINTY

As mentioned in Section III-B, total uncertainty is the sum of epistemic and aleatoric uncertainty. When the model frequently visits certain states and collects data from those states, the epistemic uncertainty should decrease, while the aleatoric uncertainty remains as data-driven uncertainty that cannot be reduced through training. However, our estimation of the aleatoric uncertainty becomes more accurate over time, allowing us to place more weight on this estimation. Hence, our total uncertainty estimation is:

$$u_T(s, a) = u_{ep}(s, a) z_\alpha + \frac{1}{1 + u_{ep}(s, a)} c(s, a) \quad (20)$$

Here, z_α is the same as the notation in (4). The case where total uncertainty is used to make decisions during the testing phase is compared in Section VI.

J. MARKET ASSUMPTION

Before conducting the experiment, we made the following assumptions:

- Zero slippage: The assets have sufficient liquidity, allowing each trade to be executed immediately at the market price when the order is placed.
- Zero market impact: The volume of trading is small enough that it does not affect the market price.

These assumptions are reasonable when trading a small number of shares relative to the average trading volume. Therefore, we can apply the single-agent off-policy offline algorithm as outlined below.

V. EXPERIMENTAL RESULT

To meet the assumptions in Section IV, we have chosen assets that have significant market capitalization and active market trade. The training and test datasets are chosen from the US market as Table 1. We considered assets with tickers of AAPL, INTC, AMZN, WMT, C, VXUS, GSPC, CL = F, GC = F, FVX and BTC in the US asset

Algorithm 1 Back Propagation Algorithm

Randomly initialize critic $Q(s, a)$ with w^Q parameters, actor network $\pi(s)$ with w^π parameters, and cost network $c(s, a)$ with w^c , VAE as $E(s), D(s)$ with weights w^e, w^d . Initialize target network Q' and π' with weights $w^{Q'} \leftarrow w^Q, w^{\pi'} \leftarrow w^\pi$. Initialize replay buffer R .

for episode = 1, M **do**

 Randomly choose initial observation date state- s_1

for $t = 1, T$ **do**

 Sample a_t from $Dir(\kappa \cdot \pi(s_t))$

 Execute action a_t then observe reward r_t and new state s_{t+1}

 Store transition (s_t, a_t, r_t, s_{t+1}) in R

 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

 Update w^e and w^d by minimizing loss:

$$Loss_{vae} = 1/N \sum_i Dis(s_i, D_{vae}(E_{vae}(s_i) \times [1; x_{x \sim N(0,1)}]^T)) + 1/N \sum_i B_{KL}(E_{vae}(s_i), N(0, 1))$$

 Set $y_i \leftarrow r_i + \gamma(1-d)Q'(s_{i+1}, \pi'(s_{i+1}))$

 Update w^Q by minimizing the loss: $L_q = 1/N \sum_i (y_i - Q(s_i, a_i))^2$

 Update λ and δ (and the weights in Dis) by minimizing the loss: $L_{ep} = 1/N \sum_i (|y_i - Q(s_i, a_i)| - u_{ep}(s_i, a_i))^2$

 Update w^c by minimizing loss: $L_c = 1/N \sum_i (VaR(s_{i+1}, a_i) - c(s_i, a_i))^2$

 Set $\phi_i \leftarrow 1/(1 + u_{ep}(s_i, a_i))$

 Update w^π by maximizing: $J_\pi = 1/N \sum_i (Q(s_i, \pi(s_i)) - C(s_i, \pi(s_i)|\phi_i))$

 Update $w^{Q'}$ and $w^{\pi'}$ by weighting:

$$w^{Q'} \leftarrow \tau w^Q + (1 - \tau)w^{Q'}$$

$$w^{\pi'} \leftarrow \tau w^\pi + (1 - \tau)w^{\pi'}$$

end for

end for

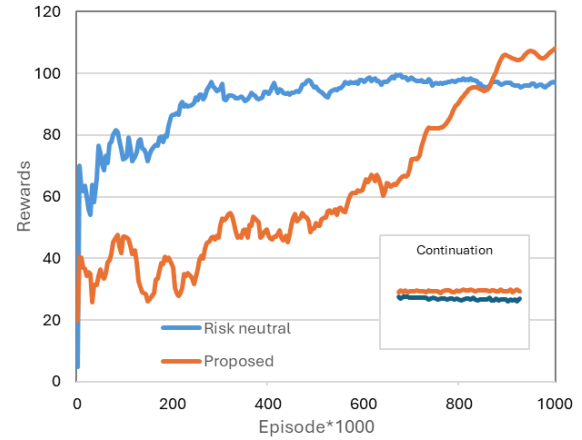
TABLE 1. Market data used for training and testing.

	Time period	Trading Days- Time steps
Training dataset	13/03/1986-20/07/2020	8963
Test dataset	21/07/2020-02/05/2024	1000

market. The selection is based upon the active trade, liquidity, industry diversification and various risk levels.

In Fig.3, the training progress is shown for both the uncertainty-aware and risk-neutral DDPG models. Initially, the uncertainty-aware model has lower average rewards during training; however, it outperforms the risk-neutral model in the long term after sufficient exploration. This is due to the added complexity of the model with the additional constraint feature and the time required to learn the dynamic risk nature of the environment.

To clarify whether overfitting occurs in epistemic uncertainty prediction, the relationships between epistemic

**FIGURE 3.** RL agent's average reward during training (vertical axis) and number of episodes (horizontal axis).

uncertainty, KL-divergence, and reconstruction error of the VAE are shown in Fig.4. There is a strong positive correlation between KL-divergence and epistemic uncertainty, and a weaker correlation between reconstruction error and epistemic uncertainty. Reference [37] shows that generative models, such as the VAE, can wrongly classify OOD data as in-distribution data. However, we do not observe such an issue in our task.

In Fig.5, we present the values of the loss functions that are discussed in the previous section. The VAE loss decreased significantly after training, though it still exhibits substantial fluctuations, likely due to the presence of out-of-distribution data. A similar pattern was observed in the training of the value network. In RL, the agent continuously performs actions to explore new or less-explored states, which RL model could not adapt to memorize yet and often results in considerable oscillations in the value network's loss. Hence, the loss values of the approximator are aptly overlooked. In our case, the loss is generally decreasing due to experience buffer, which contains both old and new data. Even though the experience replay receives OOD data to store in it, the loss on in-distributional data results in falling average loss. Notably, the prediction of epistemic uncertainty steadily decreased and eventually stabilized after training, indicating that both the VAE and value networks were efficiently trained.

In Fig.5, we present the values of the loss functions discussed in the previous section. The VAE loss decreased significantly after training, though it still exhibits substantial fluctuations, likely due to the presence of OOD data. A similar pattern was observed in the training of the value network. In RL, the agent continuously performs actions to explore new or less-explored states, which the RL model has not yet adapted to memorize, often resulting in considerable oscillations in the value network's loss. Hence, the loss values of the approximator are typically overlooked. In our case, the loss generally decreases due to the experience buffer, which contains both old and new data. Although the experience

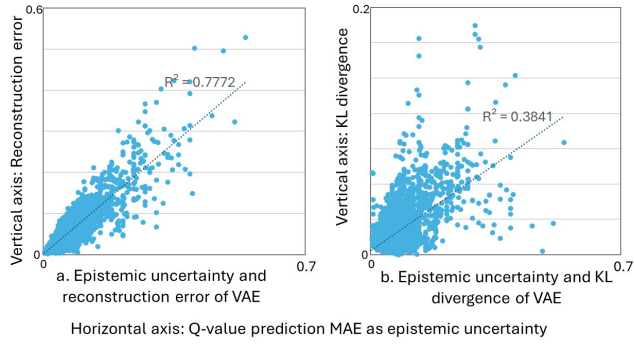


FIGURE 4. Relationship between epistemic uncertainty and prediction variables.

replay stores OOD data, the loss on in-distribution data leads to a decreasing average loss. Notably, the prediction of epistemic uncertainty steadily decreased and eventually stabilized after training, indicating that both the VAE and value networks were efficiently trained.

Figure 6 compares the proposed RL model's performance with other state-of-the-art RL algorithms such as Proximal Policy optimization (PPO) [38], Trust Region Policy Optimization (TRPO) [39] and Bayesian Policy Gradient [40]. Each model is trained on the same training dataset and tested under the same conditions. The objective for the comparable models is to maximize the Sharpe ratio to account for risk-aversion, making it appropriate to compare risk-averse models against one another. Therefore, the risk-adjusted performance metric—Sharpe ratio—is used to compare the portfolios. Over the same 1000 steps (days) of the test period, our algorithm outperformed the other models by the end of the testing phase and sometimes showed better performance during the period, except for PPO. We selected the mode of the Dirichlet distribution as the greedy policy during testing, as outlined in Section IV.

Table 2 compares portfolio performance metrics across various trading strategies. The proposed method demonstrates relatively high return performance compared to other strategies. However, meeting the risk constraint is the primary objective of our task. Most importantly, the proposed method did not violate the investor's risk constraint. Therefore, we conclude that our model outperformed the others, as our primary focus was on eliminating constraint breaches, with profitability being a secondary objective.

VI. ABLATION STUDY

The ablation study is shown in Table 3. The following cases are considered:

- Case1: The epistemic uncertainty parameter is not reflected in objective function. Therefore, the confidence parameter in (14) becomes 1. Even though the constraint is not violated on this case, the return is significantly reduced in Table 3. Due to OOD data, VaR estimation can be overestimated and agent could not proper exploration in this case.

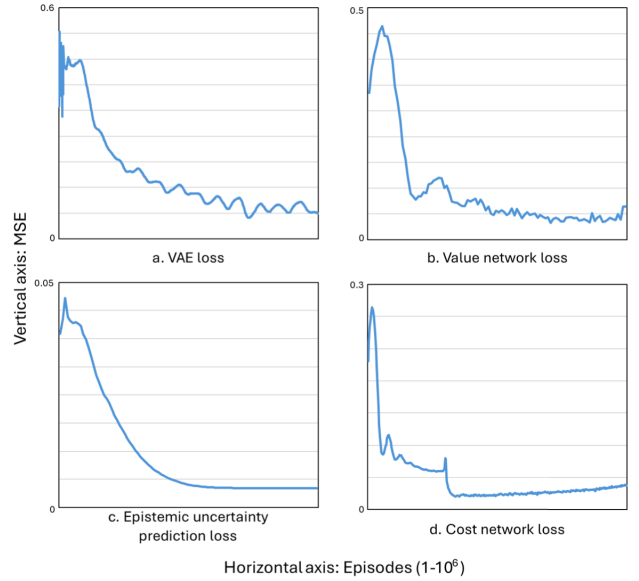


FIGURE 5. Training losses of networks in the model.

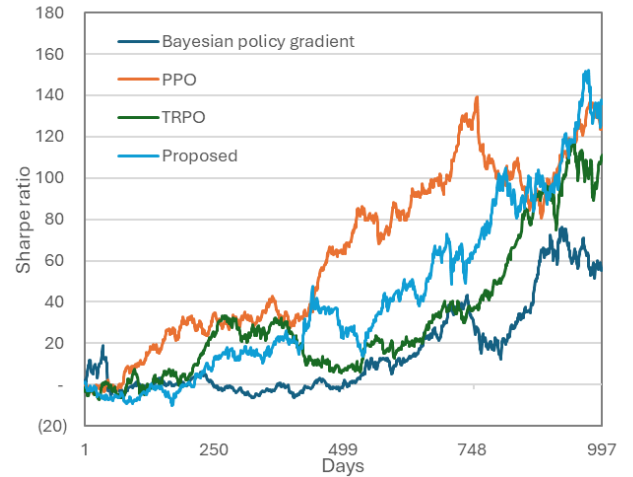


FIGURE 6. Performance of portfolios constructed using different RL algorithms.

- Case2: The constraint violation penalty term (14) is not included in objective function (15). This case becomes similar to risk-neutral RL model. Hence, the constraint is violated and the return is increased on the test dataset in Table 3.
- Case3: Weighted Euclidean distance is used to assess the reconstruction error, because the different inputs might have nonidentical impact on the epistemic uncertainty. However, these weights have changed a little and result is almost indifferent in Table 3.
- Case4: Total uncertainty exceeded the preset constraint is reflected and the actor's action is replaced with the current asset weights in the portfolio. When the uncertainty is high, the asset weights remain the same to discourage adhering the highly uncertain decision from

TABLE 2. Performance comparison with selected metrics (rows) and different portfolio construction methods (columns) on the test dataset.

	Traditional Markowitz ^a	Proximal Policy Optimization	Trust Region Policy Optimization	Bayesian policy gradient	Proposed method
Annualized return	20.3%	30.5%	23.5%	24.8%	28.6%
Daily average VAR ¹	3.5%	5.8%	4.5%	3.2%	4.0%
Sharpe ratio-annualized ²	30.2%	36.4%	30.8%	19.4%	34.0%
Maximum drawdown ³	66%	81%	65%	51%	59%
Sortino ratio ⁴	1.76	1.32	1.41	1.72	1.68
Number of constraint violation	0	57	26	2	0

¹Based on historical mean and variance. ²Sharpe ratio = (Average return - Treasury yield) / Std.deviation. ³Maximum drawdown = (Max value - Min value) / Max value over last 60 days. ⁴Sortino ratio = (Average return - Treasury yield) / Std.deviation of negative changes.

^aBased on historical mean and variance and used Genetic algorithm to optimize.

TABLE 3. Ablation performance.

	Performance Ratio of Annual return ¹	Number of Constraint violation
Case1: Absence of Epistemic uncertainty parameter in objective function	0.85	0
Case2: Absence of Constraint violation penalty term in objective function	1.13	32
Case3: Learnable Euclidean distance parameters in epistemic uncertainty	0.99	0
Case4: Total uncertainty exceeded the constraint is reflected in testing phase	0.87	0

¹The proposed model's performance is regarded as 1.00, and other cases are compared by ratio: other case performance/proposed model performance

the actor. The performance has not been improved at all, and this case can be favourable to only highly-risk averse investors.

VII. FUTURE WORKS

Possible limitation of the model is discussed below:

- **Model complexity:** The use of cost network adds complexity to the model. The computational cost is higher compared to simpler models. In addition, convergence time of the proposed model is slower. Therefore, adding other means for faster convergence can be developed.
- **Exploration-Exploitation Strategy:** The exploration strategy is essential in RL. Safe model discourages explorativeness of the model, hence it could result slow convergence. In the future, it is essential to add estimation of risk arising from uncertainties due to unexplored states.
- **Direct risk comparison between models:** The proposed model estimates portfolio uncertainties as risk. However, the existing benchmark models utilize historical variance as risk. Therefore, direct comparison between riskiness of the models could not be ideal. Therefore, building same comparable measures in the models is important.

VIII. CONCLUSION

In this paper, we proposed an uncertainty-averse reinforcement learning algorithm for optimal portfolio allocation. Our approach effectively reduced risk during the agent's testing phase, demonstrating its efficacy. We concluded that incorporating risk-averse RL techniques is a promising strategy for portfolio optimization, particularly for risk-averse investors. Future research could explore additional performance metrics and dimensions, as well as integrate factors such as news sentiment, financial statements, and other economic indicators to further enhance performance. By incorporating more features, we could also reduce uncertainty. Our results show that the proposed risk-averse portfolio optimization approach outperforms traditional methods, even with a limited feature set.

REFERENCES

- [1] J. W. Lee, "Stock price prediction using reinforcement learning," in *Proc. IEEE Int. Symp. Ind. Electron. (ISIE)*, vol. 1, Jul. 2001, pp. 690–695.
- [2] V. Konda and J. Tsitsiklis, "Actor-critic algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 12, 1999, pp. 1–7.
- [3] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher, M. Shahzad, W. Yang, R. Bamler, and X. X. Zhu, "A survey of uncertainty in deep neural networks," *Artif. Intell. Rev.*, vol. 56, no. 1, pp. 1513–1589, Jul. 2023.
- [4] Z. Huang, H. Lam, and H. Zhang, "Quantifying epistemic uncertainty in deep learning," 2021, *arXiv:2110.12122*.
- [5] J. Van Amersfoort, L. Smith, Y. W. Teh, and Y. Gal, "Uncertainty estimation using a single deep deterministic neural network," in *Proc. Int. Conf. Mach. Learn.*, vol. 119, Jul. 2020, pp. 9690–9700.
- [6] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1050–1059.
- [7] H. Krasowski, J. Thumm, M. Müller, L. Schäfer, X. Wang, and M. Althoff, "Provably safe reinforcement learning: A theoretical and experimental comparison," *Trans. Mach. Learn. Res.*, 2023, doi: 10.48550/arXiv.2205.06750.
- [8] A. Tamar, Y. Glassner, and S. Mannor, "Optimizing the cvar via sampling," in *Proc. AAAI Conf. Artif. Intell.*, vol. 29, 2015, pp. 1–7.
- [9] I. Greenberg, Y. Chow, M. Ghavamzadeh, and S. Mannor, "Efficient risk-averse reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 32639–32652.
- [10] O. Mihatsch and R. Neuneier, "Risk-sensitive reinforcement learning," *Mach. Learn.*, vol. 49, pp. 267–290, Sep. 1998. [Online]. Available: <https://api.semanticscholar.org/CorpusID:29789746>
- [11] B. Enkhsaikhan and O. Jo, "Risk-averse reinforcement learning for portfolio optimization," *ICT Exp.*, vol. 10, no. 4, pp. 857–862, Aug. 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S240595952400047X>

- [12] Y. Chow, M. Ghavamzadeh, L. Janson, and M. Pavone, "Risk-constrained reinforcement learning with percentile risk criteria," *J. Mach. Learn. Res.*, vol. 18, no. 167, pp. 1–51, 2018.
- [13] J. B. Heaton, N. G. Polson, and J. H. Witte, "Deep learning for finance: Deep portfolios," *Appl. Stochastic Models Bus. Ind.*, vol. 33, no. 1, pp. 3–12, Jan. 2017. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/asmb.2209>
- [14] W. Bao, J. Yue, and Y. Rao, "A deep learning framework for financial time series using stacked autoencoders and long-short term memory," *PLoS ONE*, vol. 12, no. 7, Jul. 2017, Art. no. e0180944, doi: [10.1371/journal.pone.0180944](https://doi.org/10.1371/journal.pone.0180944).
- [15] Y. Shen, R. Huang, C. Yan, and K. Obermayer, "Risk-averse reinforcement learning for algorithmic trading," in *Proc. IEEE Conf. Comput. Intell. Financial Eng. Econ. (CIFER)*, Mar. 2014, pp. 391–398.
- [16] Z. Jiang, D. Xu, and J. Liang, "A deep reinforcement learning framework for the financial portfolio management problem," 2017, *arXiv:1706.10059*.
- [17] J. Moody, L. Wu, Y. Liao, and M. Saffell, "Performance functions and reinforcement learning for trading systems and portfolios," *J. Forecasting*, vol. 17, no. 56, pp. 441–470, Sep. 1998.
- [18] R. Rebonato and A. Denev, "Coherent asset allocation and diversification in the presence of stress events," *SSRN Electron. J.*, vol. 10, no. 4, pp. 1–36, 2011, doi: [10.2139/ssrn.1824207](https://doi.org/10.2139/ssrn.1824207).
- [19] H. Markowitz, "Portfolio selection," *J. Finance*, vol. 7, no. 1, pp. 77–91, 1952. [Online]. Available: <http://www.jstor.org/stable/2975974>
- [20] E. Hüllermeier and W. Waegeman, "Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods," *Mach. Learn.*, vol. 110, no. 3, pp. 457–506, Mar. 2021, doi: [10.1007/s10994-021-05946-3](https://doi.org/10.1007/s10994-021-05946-3).
- [21] Z. Xiao, Q. Yan, and Y. Amit, "Likelihood regret: An out-of-distribution detection score for variational auto-encoder," in *Proc. 34th Neural Inf. Process. Syst.*, 2020, pp. 20685–20696.
- [22] A. Kendall and Y. Gal, "What uncertainties do we need in Bayesian deep learning for computer vision?" in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Red Hook, NY, USA: Curran Associates, 2017, pp. 5574–5584.
- [23] T. Fauvel and M. Chalk, "Efficient exploration in binary and preferential Bayesian optimization," 2021, *arXiv:2110.09361*.
- [24] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, *arXiv:1312.6114*.
- [25] J. An and S. Cho, "Variational autoencoder based anomaly detection using reconstruction probability," in *Proc. Special Lect. IE*, 2015, pp. 1–18.
- [26] H. Choi, E. Jang, and A. A. Alemi, "WAIC, but why? Generative ensembles for robust anomaly detection," 2019, *arXiv:1810.01392*.
- [27] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
- [28] T. Cui, S. Ding, H. Jin, and Y. Zhang, "Portfolio constructions in cryptocurrency market: A CVaR-based deep reinforcement learning approach," *Econ. Model.*, vol. 119, Feb. 2023, Art. no. 106078.
- [29] N. A. Urpí, S. Curi, and A. Krause, "Risk-averse offline reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, 2021. [Online]. Available: <https://openreview.net/forum?id=TB1zh9b5eaz>
- [30] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [31] S. B. Achelis, *Technical Analysis From A To Z*. New York, NY, USA: McGraw-Hill, 2013.
- [32] E. Gately, *Neural Networks for Financial Forecasting*. Hoboken, NJ, USA: Wiley, 1995.
- [33] C. J. Watkins and P. Dayan, "Technical note: Q-learning," *Mach. Learn.*, vol. 8, no. 3, pp. 279–292, May 1992, doi: [10.1023/a:1022676722315](https://doi.org/10.1023/a:1022676722315).
- [34] Z. Wang, V. Bapst, N. Heess, V. Mnih, R. Munos, K. Kavukcuoglu, and N. de Freitas, "Sample efficient actor-critic with experience replay," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1875–1894. [Online]. Available: <https://openreview.net/forum?id=HyM25Mqel>
- [35] B. X. Yong and A. Brintrup, "Bayesian autoencoders with uncertainty quantification: Towards trustworthy anomaly detection," *Expert Syst. Appl.*, vol. 209, Dec. 2022, Art. no. 118196. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417422013562>
- [36] Y. Tian, M. Han, C. Kulkarni, and O. Fink, "A prescriptive Dirichlet power allocation policy with deep reinforcement learning," *Rel. Eng. Syst. Saf.*, vol. 224, Aug. 2022, Art. no. 108529. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0951832022001831>
- [37] E. T. Nalisnick, A. Matsukawa, Y. W. Teh, D. Görür, and B. Lakshminarayanan, "Do deep generative models know what they don't know?" in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 954–972.
- [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [39] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1889–1897.
- [40] H. Eriksson and C. Dimitrakakis, "Epistemic risk-sensitive reinforcement learning," in *Proc. Eur. Symp. Artif. Neural Netw. (ESANN), Comput. Intell. Mach. Learn.*, Oct. 2020, pp. 339–344. [Online]. Available: <https://www.esann.org/sites/default/files/proceedings/2020/ES2020-84.pdf>



interest is facilitating AI systems in financial applications.



BAYARAA ENKHSAIKHAN (Student Member, IEEE) received the bachelor's degree in banking and finance from Mongolian National University, in 2011, and the master's degree in finance field from The University of Melbourne, Australia, in 2019. He is currently pursuing the Ph.D. degree in computer science with Chungbuk National University, South Korea. He was with finance and banking industry, from 2011 to 2018, progressing from a Risk Manager to a Senior Staff position. His

OHYUN JO (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electrical engineering from Korea Advanced Institute of Science and Technology (KAIST), in 2005, 2007, and 2011, respectively. From April 2011 to February 2016, he was with Samsung Electronics, in charge of research and development for future wireless communication systems, applications, and services. From March 2016 to July 2017, he was a Senior Researcher with the Electronics and Telecommunications Research Institute (ETRI). From August 2017 to February 2018, he was an Associate Professor with the Department of Electrical Engineering, Korea Military Academy. He is currently an Associate Professor with the Department of Computer Science, Chungbuk National University. He has authored or co-authored more than 60 articles. He holds more than 160 registered and filed patents. His research interests include light-weight machine learning model, applications in the field of machine learning and artificial intelligence, millimeter-wave communications, next-generation WLAN/WPAN systems, 5G/6G mobile communication systems, military communications, the Internet of Things, future wireless solutions/applications/services, and embedded communications ASIC design. He was a recipient of several recognition, including the Gold Prize in Samsung Annual Award, the Most Creative Researcher of the Year Award, the Best Mentoring Award, the Major Research and Development Achievement Award, the Best Improvement of Organization Culture Award, and the Best Faculty of the Year Awards by Chungbuk National University.

• • •